🎨 Онбординг: фронтенд

Ты работаешь со статическим фронтендом: HTML, CSS, JS. **Никаких сборщиков, npm, node_modules** — всё максимально просто. Мы осознанно не усложняем, чтобы сосредоточиться на логике проекта.

1. Что должно быть установлено

- **Python 3.11+** — да, для фронта тоже. Он нужен только как **статический HTTP-сервер** (одна команда `python -m http.server`). Ставить что-то ещё не надо.
- **VS Code** или другой редактор (по желанию — см. альтернативу в п.3).
- **Git** — у тебя уже есть.

Проверь: `python --version` (на Windows) или `python3 --version` (на macOS/Linux) — должно быть 3.11 или новее.

> 💡 Если Python не хочется ставить вообще — см. альтернативу через расширение VS Code в п.3. Но Python-вариант надёжнее.

2. Почему нельзя открыть `index.html` двойным кликом

Если дважды кликнуть по `index.html`, браузер откроет `file:///…/index.html`. Это **не работает** для нашего проекта:

- `fetch()` к бэкенду (`http://127.0.0.1:8000/...`) заблокируется браузером по CORS.
- `<script type="module">` и `import` из JS-файлов не загрузятся.
- Относительные пути к ресурсам могут вести себя странно.

Правило: фронт всегда запускается через локальный HTTP-сервер. Это одна команда — ниже.

3. Как запустить фронт

Windows — самый простой способ: Дважды кликни по `serve.bat` в папке `frontend/`. Откроется окно консоли с сообщением `Starting static server at http://127.0.0.1:3000`. Открой в браузере:

http://127.0.0.1:3000

Не закрывай окно консоли, пока работаешь. Закрыл — сервер остановился. Ошибки в консоли видно там же.

macOS / Linux:

```
cd frontend
./serve.sh
```

Или вручную (любая ОС):

```
cd frontend
python -m http.server 3000 --bind 127.0.0.1
```

(на macOS/Linux — `python3` вместо `python`)

Альтернатива без Python — расширение VS Code:

- Установи расширение **Live Server** от Ritwick Dey.
- Открой `frontend/index.html` в VS Code, нажми правой кнопкой → **Open with Live Server**.
- Откроется на порту 5500 (или другом). Это тоже подходит — главное, что через HTTP, а не `file://` .

4. Как устроен проект

```
frontend/
├── index.html          ← точка входа, открывается по адресу http://127.0.0.1:3000/
├── css/
│   └── style.css       ← все стили
├── js/
│   └── main.js         ← твой JS-код
├── assets/             ← картинки, шрифты, звуки
├── serve.bat           ← запуск сервера на Windows (двойной клик)
└── serve.sh            ← запуск на macOS/Linux
```

Корень сервера — папка `frontend/` . Значит:

- `http://127.0.0.1:3000/` → `frontend/index.html`
- `http://127.0.0.1:3000/css/style.css` → `frontend/css/style.css`
- `http://127.0.0.1:3000/assets/logo.png` → `frontend/assets/logo.png`

Как подключать файлы в `index.html` :

```
<link rel="stylesheet" href="css/style.css">
<script type="module" src="js/main.js"></script>
```

Пути — относительные (не `/css/style.css` с ведущим слэшем). Тогда структура будет переносима, если позже отдадим фронт через другой префикс или заменим раздачу.

Обязательно `type="module"` для скрипта — это даёт `import` / `export` и изолированную область видимости. Плюс дефолтное поведение — «выполнить после парсинга DOM», так что не надо оборачивать в `DOMContentLoaded` .

5. Как это будет связано с бэкендом

- **Фронт:** `http://127.0.0.1:3000`
- **Бэк:** `http://127.0.0.1:8000` (поднимает отдельная команда, пока не наша забота)

Из JS мы будем ходить на бэк по полным адресам:

```
const res = await fetch('http://127.0.0.1:8000/api/items/');
const data = await res.json();
```

⚠️ **Про CORS.** Браузер считает `:3000` и `:8000` **разными origin**, поэтому при первом запросе к бэкенду может появиться ошибка вида:

```
Access to fetch at 'http://127.0.0.1:8000/api/items/' from origin 'http://127.0.0.1:300
has been blocked by CORS policy.
```

Это нормально и решается на стороне бэкенда — там включается пакет `django-cors-headers`. Ничего не чини со своей стороны, просто сообщи бэкенд-команде, что «прилетает CORS-ошибка». Чинится одной строкой в `settings.py`.

6. С чего начать

1. Убедись, что сервер запускается и `http://127.0.0.1:3000` открывает `index.html`.
2. Открой DevTools в браузере (F12) → вкладка **Console** — здесь будут ошибки JS.
3. Вкладка **Network** — покажет все запросы, включая `fetch` к бэкенду (очень пригодится для отладки CORS).
4. Создай `css/style.css` и `js/main.js`, подключи их в `index.html`, проверь, что стили и JS применились (например, `console.log('hello')` из `main.js` должен появиться в Console).

7. Что НЕ надо делать

- ❌ Не открывать `index.html` через `file://`.
- ❌ Не ставить `npm`, `vite`, `webpack` — пока не нужно, и это не наша задача.
- ❌ Не класть `node_modules`, `package.json` — мы их не используем.
- ❌ Не трогать папку `/backend` — там своя команда.
- ❌ Не хардкодить URL бэкенда в куче мест — заведём один конфиг-файл, когда дойдём до API (например, `js/config.js` с `export const API_BASE = 'http://127.0.0.1:8000';`).

8. Частые проблемы

- **`python: command not found`** — Python не установлен или не в PATH. Проверь `python --version` / `python3 --version`, при необходимости добавь Python в PATH.
- **`OSError: [Errno 98] Address already in use`** — порт 3000 занят. Убей старый процесс или запусти на другом порту: `python -m http.server 3001`. Но тогда помни: URL фронта тоже станет `:3001`.
- **Открывается пустая страница** — проверь, что `index.html` лежит прямо в `frontend/`, а не в подпапке. И что сервер запущен **из** `frontend/`.
- **CSS/JS «не подключаются»** — открой вкладку **Network** в DevTools, найди 404. Чаще всего

— ошибка в пути (`css/` вместо `css/` — с кириллицей) или опечатка.

- **`fetch` возвращает CORS-ошибку** — это бэкенд, см. п.5.

9. Чек-лист «я готов»

- [ ] Python 3.11+ установлен (`python --version` работает)
- [ ] Запуск `serve.bat` (или `./serve.sh` , или Live Server) открывает `http://127.0.0.1:3000`
- [ ] Я понимаю, что открывать через `file://` нельзя
- [ ] Структура `css/` , `js/` , `assets/` создана, `style.css` и `main.js` подключены через относительные пути
- [ ] `console.log('hello')` из `main.js` виден в Console DevTools
- [ ] Я знаю, что бэкенд — на `:8000` , фронт — на `:3000` , и что CORS-ошибки решает бэкенд-команда

Готово — можно брать первую UI-задачу. 🎨